

Assignment Questions:

1. Decision Control Statements:

Explain the concept of decision control statements in Python. Discuss the syntax and usage of the following:

- **if, else, and elif statements**
- **How these statements are used to execute different blocks of code based on certain conditions. Provide an example where if, else, and elif are used to categorize an integer as positive, negative, or zero.**

Decision Control Statements

Decision control statements in Python allow you to execute different blocks of code based on whether a specific condition is true or false. They provide the logic and flow within your programs.

if, else, and elif Statements

- **if:**
 - Executes a block of code only if the specified condition is **True**.
 - Syntax:

if condition:

 # Code to execute if condition is True

else:

- Executes a block of code if the **if** condition is **False**.
- Syntax:

if condition:

 # Code to execute if condition is True

else:

 # Code to execute if condition is False

elif:

- Stands for "else if."
- Used to check for multiple conditions sequentially.
- If the `if` condition is `False`, the `elif` conditions are checked in order.
- If none of the `if` or `elif` conditions are `True`, the `else` block (if present) is executed.
- Syntax:

```
if condition1:
```

```
    # Code to execute if condition1 is True
```

```
elif condition2:
```

```
    # Code to execute if condition2 is True
```

```
elif condition3:
```

```
    # Code to execute if condition3 is True
```

```
else:
```

```
    # Code to execute if none of the above conditions are True
```

Example: Categorizing an Integer

```
num = int(input("Enter an integer: "))
```

```
if num > 0:
```

```
    print(num, "is positive.")
```

```
elif num < 0:
```

```
    print(num, "is negative.")
```

```
else:
```

```
    print(num, "is zero.")
```

Decision Control Statements

Decision control statements in Python allow you to execute different blocks of code based on whether a specific condition is true or false. They provide the logic and flow within your programs.

if, else, and elif Statements

- **if:**

- Executes a block of code only if the specified condition is **True**.

Syntax:

Python
if condition:

Code to execute if condition is True

◦

- **else:**

- Executes a block of code if the **if** condition is **False**.

Syntax:

Python
if condition:

Code to execute if condition is True

else:

Code to execute if condition is False

- **elif:**

- Stands for "else if."
- Used to check for multiple conditions sequentially.
- If the **if** condition is **False**, the **elif** conditions are checked in order.
- If none of the **if** or **elif** conditions are **True**, the **else** block (if present) is executed.

Syntax:

if condition1:

```
# Code to execute if condition1 is True

elif condition2:

    # Code to execute if condition2 is True

elif condition3:

    # Code to execute if condition3 is True

else:

    # Code to execute if none of the above conditions are True
```

○

Example: Categorizing an Integer

```
num = int(input("Enter an integer: "))

if num > 0:

    print(num, "is positive.")

elif num < 0:

    print(num, "is negative.")

else:

    print(num, "is zero.")
```

In this example:

- The `if` statement checks if `num` is greater than 0. If `True`, it prints that the number is positive.
- If the `if` condition is `False`, the `elif` statement checks if `num` is less than 0. If `True`, it prints that the number is negative.
- If both the `if` and `elif` conditions are `False`, the `else` block¹ executes, indicating that the number is zero.

2. Looping and Nested Loops:

Discuss the difference between **while** loops and **for** loops in Python.

- Describe how both loops work and when to use each.
- Explain the concept of **nested loops** and provide an example of a program that uses a nested loop to print a multiplication table.

Loops are fundamental control flow structures that allow you to repeatedly execute a block of code. Python provides two primary types of loops:

1. While Loops

- **How they work:**
 - A **while** loop continues to execute as long as a given condition remains **True**.
 - The condition is checked before each iteration of the loop.
- **Syntax:**

```
while condition:

    # Code to be executed repeatedly
```
- **When to use:**
 - When you don't know the exact number of iterations in advance.
 - When the loop's termination depends on a dynamic condition that changes during execution.

2. For Loops

- **How they work:**
 - **for** loops iterate over a sequence (like a list, tuple, string, or range) of elements.
 - In each iteration, the loop variable takes on the value of the current element.
- **Syntax:**

```
for item in sequence:

    # Code to be executed for each item
```
- **When to use:**
 - When you need to iterate over a known sequence of elements.
 - When you want to execute a block of code a specific number of times.

Nested Loops

- **Concept:** Nested loops occur when one loop is placed inside another loop.
- **How they work:** The inner loop executes completely for each iteration of the outer loop.

- **Example: Multiplication Table**

```
for i in range(1, 11):  
    for j in range(1, 11):  
        print(i * j, end=" ")  
  
    print()
```

Looping in Python

Loops are fundamental control flow structures that allow you to repeatedly execute a block of code. Python provides two primary types of loops:

1. While Loops

- **How they work:**
 - A **while** loop continues to execute as long as a given condition remains **True**.
 - The condition is checked before each iteration of the loop.

Syntax:

```
Python  
while condition:  
  
    # Code to be executed repeatedly
```

-
- **When to use:**
 - When you don't know the exact number of iterations in advance.
 - When the loop's termination depends on a dynamic condition that changes during execution.

2. For Loops

- **How they work:**
 - **for** loops iterate over a sequence (like a list, tuple, string, or range) of elements.
 - In each iteration, the loop variable takes on the value of the current element.

Syntax:

Python
for item in sequence:

Code to be executed for each item

-
- **When to use:**
 - When you need to iterate over a known sequence of elements.
 - When you want to execute a block of code a specific number of times.

Nested Loops

- **Concept:** Nested loops occur when one loop is placed inside another loop.
- **How they work:** The inner loop executes completely for each iteration of the outer loop.
- **Example: Multiplication Table**

Python

```
for i in range(1, 11):  
  
    for j in range(1, 11):  
  
        print(i * j, end=" ")  
  
    print()
```

This code will print a multiplication table from 1 to 10.

- **Explanation:**
 - The outer loop iterates from 1 to 10, representing the rows.
 - For each row, the inner loop iterates from 1 to 10, representing the columns.
 - Inside the inner loop, the product of `i` and `j` is calculated and printed.
 - After the inner loop completes, a newline character is printed to move to the next row.

Nested loops are useful for tasks that involve iterating over multiple dimensions, such as working with matrices or generating complex patterns.

3. List, Dictionary, and Set Comprehensions:

Define **comprehensions** in Python.

- Explain **list comprehensions**, **dictionary comprehensions**, and **set comprehensions**.
- Discuss their syntax, how they can be used to create new collections in a concise way, and provide examples to generate:
 - A list of squares of even numbers between 1 and 10
 - A dictionary of square roots of integers from 1 to 5
 - A set containing only unique letters from a string

Comprehensions in Python

Comprehensions are a concise and elegant way to create lists, dictionaries, and sets in Python. They provide a more readable and efficient alternative to traditional loop-based approaches for constructing these data structures.

1. List Comprehensions

- **Syntax:**

[expression for item in iterable if condition]

Explanation:

- **expression**: The value to be included in the new list.
- **item**: A variable representing each element in the iterable.
- **iterable**: Any iterable object (e.g., list, tuple, string, range).
- **condition** (optional): A boolean expression that filters elements from the iterable.

Example: Squares of even numbers

```
even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]
```

```
print(even_squares) # Output: [4, 16, 36, 64, 100]
```

Explanation:

- **key**: The key for the dictionary entry.
- **value**: The corresponding value for the key.
- **item**: A variable representing each element in the iterable.
- **iterable**: Any iterable object.
- **condition** (optional): A boolean expression that filters elements from the iterable.

Example: Square roots of integers

```
import math # Import math module for square root function
```

```
sqrt_dict = {x: math.sqrt(x) for x in range(1, 6)}
```

```
print(sqrt_dict) # Output: {1: 1.0, 2: 1.4142135623730951, 3: 1.7320508075688772, 4: 2.0, 5: 2.23606797749979}
```

Set Comprehensions

- **Syntax:**

```
{expression for item in iterable if condition}
```

Explanation:

- **expression:** The value to be included in the set.
- **item:** A variable representing each element in the iterable.
- **iterable:** Any iterable object.
- **condition** (optional): A boolean expression that filters elements from the iterable.

Example: Unique letters from a string

```
text = "hello world"
```

```
unique_letters = {char for char in text}
```

```
print(unique_letters) # Output: {' ', 'd', 'e', 'h', 'l', 'o', 'r', 'w'}
```

Comprehensions provide a concise and efficient way to create new collections in Python, making your code more readable and easier to maintain.

4. Arrays: 1D and 2D Arrays in Python:

Explain the concept of **arrays** in Python, focusing on:

- How arrays differ from lists in Python
- How to create and manipulate 1D arrays using the **array** module
- How to work with 2D arrays using the **numpy module**
- Provide examples to demonstrate how to create, initialize, and access elements in both 1D and 2D arrays.

Arrays in Python

Arrays in Python are specialized data structures that store a collection of elements of the same data type. They are similar to lists, but with some key differences:

- **Data Type Homogeneity:**
 - **Lists:** Can store elements of different data types (e.g., integers, floats, strings).
 - **Arrays:** Can only store elements of the same data type. This restriction allows for more efficient memory usage and faster operations.
- **Performance:**
 - Arrays, especially those created using the `array` module, can be more memory-efficient and perform faster for certain operations compared to lists, particularly when dealing with large amounts of numerical data.

1D Arrays (using the `array` module)

- **Creating a 1D array:**

```
import array as arr

# Create an array of integers

my_array = arr.array('i', [1, 2, 3, 4, 5])

print(my_array)
```

Accessing elements:

```
first_element = my_array[0] # Access the first element
print(first_element)
last_element = my_array[-1] # Access the last element
print(last_element)
```

Modifying elements:

```
my_array[2] = 10 # Modify the third element
print(my_array)
```

2D Arrays (using the `numpy` module)

- **Creating a 2D array:**

```
import numpy as np

# Create a 2D array with 3 rows and 4 columns
my_2d_array = np.array([[1, 2, 3, 4],
```

```
[5, 6, 7, 8],  
[9, 10, 11, 12]])  
print(my_2d_array)
```

Accessing elements:

```
element = my_2d_array[1, 2] # Access the element at row 1, column 2  
print(element)
```

Modifying elements:

```
my_2d_array[0, 0] = 100 # Modify the element at row 0, column 0  
print(my_2d_array)
```

Key Points:

- The **array** module is suitable for 1D arrays of homogeneous data types.
- The **numpy** module is powerful for working with multidimensional arrays and provides a wide range of array operations.
- Choose the appropriate data structure based on the specific requirements of your application.

This explanation provides a basic understanding of arrays in Python, their differences from lists, and how to work with 1D and 2D arrays using the **array** and **numpy** modules.

5. Functions and Recursion:

Describe the importance of **functions** in Python.

- Explain how functions help in code modularization and reuse.
- Discuss function parameters, return values, and scope of variables within functions.
- Define **recursion** and explain how recursive functions work, providing an example of a function to compute the factorial of a number.

Functions in Python

Importance of Functions

- **Code Organization and Modularity:**
 - Functions break down complex problems into smaller, more manageable units.
 - This improves code readability, maintainability, and reduces complexity.
- **Code Reusability:**
 - Once defined, a function can be called multiple times throughout your program.
 - This avoids code duplication, making your code more concise and easier to update.

- **Abstraction:**

- Functions encapsulate a specific task, allowing you to use them without needing to know the internal details of their implementation.

Key Concepts

- **Parameters:**

- Values passed to a function when it is called.
- Allow you to provide input to the function.

- **Return Values:**

- The output produced by the function.
- A function can return a single value or multiple values using tuples.

- **Scope:**

- Determines the visibility of variables within a program.
- Variables defined inside a function have local scope, meaning they are only accessible within that function.
- Variables defined outside a function have global scope and can be accessed from anywhere in the program.
-

Recursion

- **Definition:**

- A technique where a function calls itself directly or indirectly.
- Recursive functions must have a base case (a condition that stops the recursion) to prevent infinite loops.

- **How it Works:**

- The function breaks down a problem into smaller, similar subproblems.
- Each subproblem is solved recursively until the base case is reached.
- The results of the subproblems are then combined to solve the original problem.

- **Example: Factorial of a Number**

```
def factorial(n):
```

```
    if n == 0:
```

```
        return 1 # Base case
```

```
    else:
```

```
return n * factorial(n-1)
```

Example usage:

```
result = factorial(5)
```

```
print("Factorial of 5:", result) # Output: Factorial of 5: 120
```

In this example:

- `factorial(n)` calculates the factorial of `n`.
- If `n` is 0, it returns 1 (base case).
- Otherwise, it recursively calls itself with `n-1` and multiplies the result by `n`.

Recursion can provide elegant solutions to certain problems, but it's important to be mindful of potential issues like stack overflow if the recursion depth becomes too large.

Functions are essential building blocks in Python, promoting code organization, reusability, and readability. Recursion, while sometimes less efficient than iterative approaches, can offer elegant solutions to certain problems when used appropriately.

6. Lambda Functions:

Define **lambda functions** in Python.

- Explain how they differ from regular functions defined using the `def` keyword.
- Discuss their syntax, when to use them, and provide an example that demonstrates a lambda function to square a number.

Lambda Functions

- **Definition:**
 - Lambda functions are small, anonymous functions defined using the `lambda` keyword.
 - They are often used for short, concise operations.
- **Difference from Regular Functions:**
 - **Syntax:**
 - Lambda functions have a simpler syntax: `lambda arguments: expression`

- Regular functions use the `def` keyword and have a more structured block of code.
- **Purpose:**
 - Lambda functions are typically used for short, single-line operations.
 - Regular functions can be more complex and contain multiple lines of code.
- **Name:**
 - Lambda functions don't have a name, hence "anonymous."
 - Regular functions have a name that can be used to refer to them.

Syntax:

lambda arguments: expression

- **arguments:** A comma-separated list of arguments (optional).
- **expression:** The single expression that the function evaluates and returns.

Example: Squaring a Number

```
square = lambda x: x**2
```

```
result = square(5)
print(result) # Output: 25
```

In this example:

- `lambda x: x**2` defines a lambda function that takes one argument `x` and returns its square (`x**2`).
-
- The lambda function is assigned to the variable `square`.
- `square(5)` calls the lambda function with the argument 5, and the result (25) is printed.

When to Use Lambda Functions:

- **Short, Simple Operations:** When you need a quick, one-line function for a specific task.
- **Callbacks:** As arguments to functions that expect functions as input (e.g., `filter()`, `map()`, `sorted()`).
- **Concise Code:** To make your code more concise and readable in certain situations.

Lambda functions are a powerful tool for writing concise and expressive code in Python, especially when dealing with short, functional operations.

7. Searching Techniques: Linear and Binary Search:

Explain the concepts of Linear Search and Binary Search.

- Discuss the algorithm and time complexity of both techniques.
- Describe the differences between the two, including when each search method should be used.
- Provide theoretical examples of how each search algorithm works.

Searching Techniques

1. Linear Search

- **Concept:**
 - Linear search sequentially checks each element in the list until the target element is found or the end of the list is reached.
 -
- **Algorithm:**
 - Start from the beginning of the list.
 - Compare the current element with the target element.
 - If they match, return the index of the current element.
 -
 - If they don't match, move to the next element and repeat steps 2 and 3.
 -
 - If the end of the list is reached without finding the target element, return -1 (indicating that the element is not found).
- **Time Complexity:**
 - **Best Case:** $O(1)$ (if the target element is the first element)
 -
 - **Average Case:** $O(n)$ (where 'n' is the number of elements in the list)
 -
 - **Worst Case:** $O(n)$ (if the target element is the last element or not present)
 -
- **When to Use:**
 - When the list is unsorted.
 - When the list is very small.
- **Theoretical Example:**
 - **List:** [5, 2, 9, 1, 5]
 - **Target:** 9
 - **Steps:**

1. Compare 5 with 9: Not a match.
2. Compare 2 with 9: Not a match.
3. Compare 9 with 9: Match found at index 2.

2. Binary Search

- **Concept:**

- Binary search is a more efficient search algorithm that works only on **sorted** lists.
-
- It repeatedly divides the search interval in half.
-

- **Algorithm:**

- Find the middle element of the sorted list.
- Compare the middle element with the target element.
- If the middle element is greater than the target, search the left half of the list.
- If the middle element is less than the target, search the right half of the list.
- If the middle element is equal to the target, return the index.
-
- Repeat steps 1-5 until the target element is found or the search interval becomes empty.

- **Time Complexity:**

- **Best Case:** $O(1)$ (if the middle element of the first half is the target)
- **Average Case:** $O(\log n)$
-
- **Worst Case:** $O(\log n)$
-

- **When to Use:**

- When the list is sorted.
- When the list is large.

- **Theoretical Example:**

- **Sorted List:** [1, 2, 5, 9, 15]
- **Target:** 9
- **Steps:**
 1. Middle element: 5
 2. $5 < 9$, so search the right half: [9, 15]
 3. Middle element of [9, 15]: 9
 4. $9 == 9$, target found at index 3.

Key Differences

Feature	Linear Search	Binary Search
Works	on Unsorted	lists Sorted lists
Time Complexity	$O(n)$	$O(\log n)$
Efficiency	Less efficient	More efficient
Approach	Checks each element sequentially	Divides the search interval in half

8. Sorting Techniques: Selection Sort and Bubble Sort:

Define and explain the Selection Sort and Bubble Sort algorithms.

- Describe the steps involved in each sorting algorithm.
- Compare the time and space complexities of both algorithms.
- Provide a theoretical example of how each algorithm sorts a given list of integers in ascending order.

Sorting Techniques: Selection Sort and Bubble Sort

Selection Sort

- **Definition:**
 - Selection Sort is a simple sorting algorithm that repeatedly selects the minimum (or maximum) element from the unsorted portion of the list and swaps it with the first element of the unsorted portion.
 -
- **Steps:**
 - Find the minimum element in the unsorted portion of the list.
 - Swap the minimum element with the first element of the unsorted portion.
 -
 - Repeat steps 1 and 2 for the remaining unsorted portion of the list.
 -
- **Time Complexity:**
 - **Best Case:** $O(n^2)$
 -
 - **Average Case:** $O(n^2)$
 -

- **Worst Case:** $O(n^2)$
-
- **Space Complexity:** $O(1)$ (in-place sorting)
- **Theoretical Example:**
 - **Unsorted List:** [64, 34, 25, 12, 22, 11, 90]
 - **Pass 1:**
 - Find minimum: 11
 - Swap 64 and 11: [11, 34, 25, 12, 22, 64, 90]
 - **Pass 2:**
 - Find minimum: 12
 - Swap 34 and 12: [11, 12, 25, 34, 22, 64, 90]
 - **Continue similarly until the list is sorted:** [11, 12, 22, 25, 34, 64, 90]

Bubble Sort

- **Definition:**
 - Bubble Sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until no swaps are made in an entire pass, indicating that the list is sorted.
 -
- **Steps:**
 - Compare adjacent elements.
 - If the first element is greater than the second, swap them.
 - Repeat steps 1 and 2 for all pairs of adjacent elements.
 - If no swaps were made in a pass, the list is sorted. Otherwise, repeat steps 1-3.
 -
- **Time Complexity:**
 - **Best Case:** $O(n)$ (if the list is already sorted)
 - **Average Case:** $O(n^2)$
 - **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$ (in-place sorting)

- **Theoretical Example:**

- **Unsorted List:** [64, 34, 25, 12, 22, 11, 90]
- **Pass 1:**
 - Compare 64 and 34: Swap -> [34, 64, 25, 12, 22, 11, 90]
 - Compare 64 and 25: Swap -> [34, 25, 64, 12, 22, 11, 90]
 - ... (continue comparisons and swaps)
- **Pass 2:**
 - ... (continue comparisons and swaps)
- **Continue passes until the list is sorted:** [11, 12, 22, 25, 34, 64, 90]

Comparison

- **Time Complexity:** Both have a worst-case time complexity of $O(n^2)$, making them less efficient for large datasets compared to more advanced sorting algorithms like Merge Sort or Quick Sort.
-
- **Space Complexity:** Both have a space complexity of $O(1)$, as they sort the list in-place without requiring extra memory.
- **Stability:** Selection Sort is not stable (the relative order of equal elements may change), while Bubble Sort is stable.
-

While both Selection Sort and Bubble Sort are simple to understand and implement, they are generally not the most efficient sorting algorithms for large datasets. More advanced sorting algorithms are typically preferred for better performance.

Programming Questions:

1. Decision Control Statements:

Write a Python program to determine whether a given number is **prime** or **composite** using **if**, **elif**, and **else** statements. The program should prompt the user to input a number and display whether the number is prime, composite, or neither (e.g., if the number is 1).

2. Looping and Nested Loops:

Write a Python program that prints the **Floyd's Triangle** using nested **for loops**. The program should take an integer **n** as input and print the first **n** rows of the triangle. The triangle should look like this for **n = 5**:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

3. List, Dictionary, and Set Comprehensions:

Write a Python program using **list comprehension**, **dictionary comprehension**, and **set comprehension** to:

- Create a list of all the **even numbers** between 1 and 50.
- Create a dictionary where keys are numbers from 1 to 5 and values are their **squares**.
- Create a set of **unique characters** from the string "programming".

4. Arrays: 1D and 2D Arrays in Python:

Write a Python program that:

- Creates a 1D array using the **array** module and initializes it with integers.
- Creates a 2D array using the **numpy** module and fills it with random numbers between 1 and 100.
- Accesses the element at position **(1, 2)** in the 2D array and prints it.

5. Functions and Recursion:

Write a Python program that defines a **recursive function** to compute the **factorial** of a given number **n**. Additionally, implement a non-recursive version of the factorial calculation in a separate function.

6. Lambda Functions:

Write a Python program that uses a **lambda function** to:

- Take a list of integers as input from the user.
- Use the lambda function to filter out the even numbers from the list.
- Print the list of even numbers.

7. Searching Techniques: Linear and Binary Search:

Write two separate functions:

- A **linear search** function to find an element in an unsorted list.
- A **binary search** function to find an element in a sorted list. For both functions, the program should display the index of the element if found, or a message saying the element was not found.

8. Sorting Techniques: Selection Sort and Bubble Sort:

Write a Python program that implements both **Selection Sort** and **Bubble Sort** algorithms to sort a given list of integers in **ascending order**. The program should:

- Take a list of integers as input from the user.
- Sort the list using both sorting techniques.
- Print the sorted list for both sorting algorithms.

Additional Notes for the Programming Questions:

- For **decision control statements** and **loops**, focus on using conditional and looping constructs to solve problems that require decision-making and repetitive tasks.
- **Comprehensions** allow concise ways to generate collections like lists, dictionaries, and sets, so encourage the use of comprehensions for concise and readable code.
- For **arrays and numpy**, ensure that students are comfortable with using Python's `array` module and `numpy` library for basic array manipulation.
- **Recursion** and **lambda functions** are advanced topics that focus on problem-solving efficiency and functional programming concepts.
- **Searching and sorting algorithms** help students understand algorithm design, efficiency, and comparison of different methods (linear vs binary search, selection sort vs bubble sort).

1. Write a Python program using list comprehension, dictionary comprehension, and set comprehension to:

- **Create a list of all the even numbers between 1 and 50.**
- **Create a dictionary where keys are numbers from 1 to 5 and values are their squares.**
- **Create a set of unique characters from the string "programming".**

List Comprehension

```
even_numbers = [num for num in range(1, 51) if num % 2 == 0]
```

```
print("Even numbers:", even_numbers)
```

Dictionary Comprehension

```
squares_dict = {num: num**2 for num in range(1, 6)}
```

```
print("Squares dictionary:", squares_dict)
```

Set Comprehension

```
unique_chars = {char for char in "programming"}
```

```
print("Unique characters:", unique_chars)
```

Python

List Comprehension

```
even_numbers = [num for num in range(1, 51) if num % 2 == 0]
```

```
print("Even numbers:", even_numbers)
```

Dictionary Comprehension

```
squares_dict = {num: num**2 for num in range(1, 6)}
```

```
print("Squares dictionary:", squares_dict)
```

Set Comprehension

```
unique_chars = {char for char in "programming"}
```

```
print("Unique characters:", unique_chars)
```

This code will:

Create a list of even numbers:

[num for num in range(1, 51) if num % 2 == 0]: This line creates a list comprehension.

num for num in range(1, 51): This iterates through numbers from 1 to 50.

if num % 2 == 0: This condition filters out only the even numbers.

Create a dictionary of squares:

{num: num**2 for num in range(1, 6)}: This line creates a dictionary comprehension.

num: This is the key of the dictionary.

num**2: This is the value corresponding to the key, which is the square of the key.

Create a set of unique characters:

{char for char in "programming"}: This line creates a set comprehension.

char: This represents each character in the string.

The set data structure automatically removes duplicates.

This demonstrates the concise and efficient nature of comprehensions in Python for creating lists, dictionaries, and sets.

OUTPUT

Even numbers: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50]

Squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Unique characters: {'m', 'r', 'g', 'n', 'o', 'i', 'p'}

2.Looping and Nested Loops:

Write a Python program that prints the Floyd's Triangle using nested for loops. The program should take an integer n as input and print the first n rows of the triangle. The triangle should look like this for n = 5:

1

2 3

4 5 6

7 8 9 10

11 12 13 14 15

Explanation:

1. Get input:

- The program starts by prompting the user to enter the desired number of rows for the triangle and stores it in the variable `n`.

2. Initialize counter:

- A variable `count` is initialized to 1. This variable will keep track of the numbers to be printed in the triangle.

3. Nested loops:

- The outer `for` loop iterates `n` times, representing each row of the triangle.
- The inner `for` loop iterates `i` times, where `i` is the current row number. This loop prints the numbers for each row.
- Inside the inner loop:
 - The current value of `count` is printed, followed by a space.
 - The value of `count` is incremented by 1.

4. Newline:

- After each row is printed, the `print()` function without any argument is used to move the cursor to the next line.

This program effectively generates and prints Floyd's Triangle for the given input `n`

```
def floyd_triangle(n):
```

```
    """ Prints Floyd's Triangle with 'n' rows.
```

```
    Args:
```

```
        n: The number of rows to print in the triangle. """
```

```
    count = 1
```

```
    for i in range(1, n + 1):
```

```
        for j in range(i):
```

```
            print(count, end=" ")
```

```
            count += 1
```

```
        print()
```

```
# Get input from the user
```

```
n = int(input("Enter the number of rows: "))
```

```
# Print the Floyd's Triangle
```

```
floyd_triangle(n)
```

OUTPUT:

Enter the number of rows: 6

1

2 3

4 5 6

7 8 9 10

11 12 13 14 15

16 17 18 19 20 21

This code will:

1. **Get input:** Prompt the user to enter the number of rows (`n`) for the triangle.
2. **Initialize `count`:** Set a counter variable (`count`) to 1 to keep track of the numbers in the triangle.
3. **Outer loop:** Iterate through each row of the triangle using a `for` loop.
4. **Inner loop:** Iterate through each column in the current row.
5. **Print number:** Print the current `count` value and increment it.
6. **Newline:** After printing all numbers in a row, move to the next line.

This will effectively print Floyd's Triangle with the specified number of rows.

3.3. List, Dictionary, and Set Comprehensions:

Write a Python program using list comprehension, dictionary comprehension, and set comprehension to:

- Create a list of all the even numbers between 1 and 50.
- Create a dictionary where keys are numbers from 1 to 5 and values are their squares.
- Create a set of unique characters from the string "programming".

Explanation:

1. List Comprehension:

- `[num for num in range(1, 51) if num % 2 == 0]:`
 - Creates a list of numbers from 1 to 50 using `range(1, 51)`.
 - The `if num % 2 == 0` condition filters out only the even numbers.

2. Dictionary Comprehension:

- `{num: num**2 for num in range(1, 6)}:`
 - Creates a dictionary where:
 - `num` is the key.
 - `num**2` (the square of `num`) is the corresponding value.

3. Set Comprehension:

- `{char for char in "programming"}:`
 - Creates a set of unique characters from the string "programming".
 - Sets automatically remove duplicates.

This demonstrates the concise and efficient nature of comprehensions in Python for creating lists, dictionaries, and sets.

List Comprehension

```
even_numbers = [num for num in range(1, 51) if num % 2 == 0]
```

```
print("Even numbers:", even_numbers)
```

Dictionary Comprehension

```
squares_dict = {num: num**2 for num in range(1, 6)}
```

```
print("Squares dictionary:", squares_dict)
```

Set Comprehension

```
unique_chars = {char for char in "programming"}
```

```
print("Unique characters:", unique_chars)
```